Diagnosing Hybrid Attention/Mamba Cache Over-Reservation from vLLM Planner Artifacts

Anonymous Workshop Submission

Abstract

Hybrid Attention/Mamba language models put heterogeneous cache state behind a serving interface designed largely around uniform KV pages. This mismatch can cause a planner to reserve more memory than the useful hybrid cache payload, but measuring the gap is difficult without first separating planner evidence from runtime mutation. We present Cachepawl Path C, a read-only workflow that captures vanilla vLLM cache-plan artifacts, translates them into an advisory schema, replays the planner stage on captured inputs, and reports planner-level over-reservation. In a bounded four-cell matrix for `Zyphra/Zamba2-2.7B-instruct` under `vllm==0.21.0`, the observed planner artifacts show advisory savings from 0.64 to 1.26 GiB (685,011,456 to 1,347,563,520 bytes), with overestimation ratio 1.7333734577189286. A separate deterministic RTX 3060 planner-only pack shows that a native KV/state planner model stays within 1.007x of useful bytes on three synthetic hybrid workloads, while a uniform padded planner model ranges from 2.869x to 3.934x. These are planner/advisory results, not runtime cache substitution or serving-performance measurements.

1 Introduction

Serving systems increasingly need to plan memory for models whose internal state is not uniform. Transformer-serving systems popularized page-based KV cache management because each attention layer grows with the sequence and can be handled through a common block abstraction. Hybrid Attention/Mamba models break that simplifying assumption. Attention layers still consume KV pages, but Mamba or SSM layers carry state tensors with a different shape and reuse contract. If a serving planner accounts for both through one padded page shape, reserved memory can diverge from useful cache payload.

The systems question is not only whether a better allocator might exist. Before runtime integration is meaningful, we need to know whether the planner inputs and cache artifacts already expose the mismatch. A diagnostic workflow should answer three narrower questions. First, can a vanilla serving run be observed without mutating scheduler, allocator, or worker state? Second, can the cache plan be translated into a stable schema that separates useful bytes from reserved bytes? Third, does planner replay agree with the runtime scheduler configuration closely enough to support an advisory comparison?

This paper answers those questions for a bounded vLLM Path C evidence pack. The result is intentionally narrower than a serving-system replacement. It is a workshop-scale diagnostic study of planner artifacts for hybrid cache layouts, with an explicit boundary between advisory evidence and future controlled substitution. That boundary makes the contribution more useful, not less: it identifies what can be concluded from committed artifacts today and what must be measured before stronger runtime claims are appropriate.

Contributions. This paper makes four contributions:

- an observe/translate/replay/diagnose workflow for read-only analysis of vLLM hybrid cache-plan artifacts;
- a bounded Path C matrix showing planner-level over-reservation for one `Zyphra/Zamba2-2.7B-instruct` setup;
- a deterministic RTX 3060 planner-only comparison showing the gap between a uniform padded planner model and a native KV/state planner model;
- a compact artifact and claim boundary that keeps runtime substitution, serving memory savings, latency, throughput, and quality in future work.

2 Background and Related Work

Paged serving systems. PagedAttention and vLLM introduced a virtualized KV-cache view that lets serving systems manage transformer KV memory efficiently at request granularity [2]. SGLang targets efficient execution of structured language model programs and provides another planner-facing serving context [4]. This paper uses the same systems lens: cache planning is a concrete runtime decision, and planner artifacts can be studied directly.

Hybrid architectures. Jamba and Zamba2 combine attention with Mamba-style sequence modeling [3, 1]. These models motivate heterogeneous cache accounting because attention KV pages and Mamba state tensors do not share a single natural memory shape. The issue is not simply larger models; it is mixed state with different byte geometry.

Diagnostics before substitution. The closest framing for this paper is artifact-based systems diagnosis. Rather than replacing a runtime allocator, Cachepawl Path C asks what can be inferred from a captured planner state. This is a lower-risk step toward integration: planner diagnostics can show over-reservation in an observed plan, but they do not establish live VRAM reduction, request-admission improvement, latency, throughput, or model quality.

Len	Util.	Reserved GiB	Useful GiB	Savings GiB
2048	0.6	1.76	1.02	0.75
2048	0.7	2.97	1.71	1.26
4096	0.6	1.51	0.87	0.64
4096	0.7	2.71	1.56	1.15

Table 1: Path C advisory matrix for `Zyphra/Zamba2-2.7B-instruct`. Across these cells the overestimation ratio is 1.7333734577189286 and the wasted fraction is 0.4230902777777778.

3 Method

Figure 1 shows the Path C workflow. The design principle is that all measurements come from artifacts and replay, not mutation.

3.1 Observe

The observed run uses `Zyphra/Zamba2-2.7B-instruct` with `vllm==0.21.0`. The observation phase records the runtime cache plan, planner metadata, scheduler and KV-manager structure, worker tensor layout, request-to-block assignment, and attention block-table metadata. It does not edit vLLM source, monkeypatch runtime methods, replace allocators, or return Cachepawl plans to vLLM.

3.2 Translate and Replay

The translation step maps the observed cache plan into fields needed for advisory accounting: cache group count, cache tensor count, layer count, block count, per-group cache kind, page size, block size, useful bytes, and observed reserved bytes. The replay step reruns vLLM's planner configuration routine on captured planner inputs. In the committed evidence, replay matches the runtime scheduler cache configuration and records that runtime state did not change during replay.

3.3 Diagnose

The diagnostic report computes five planner-level quantities: reserved bytes, useful bytes, advisory savings bytes, overestimation ratio, and wasted fraction. For a plan with reserved bytes R and useful bytes U, the reported ratio is R/U, and the wasted fraction is $(R-U)/R$. These metrics are deliberately planner-facing. They quantify over-reservation in the observed or modeled planner state.

4 Evaluation

The evaluation has two tracks. The first uses observed vLLM planner artifacts for a bounded matrix over one hybrid model. The second uses a deterministic planner-only comparison over synthetic hybrid workloads for an RTX 3060 target profile. Both tracks measure planner estimates.

4.1 Observed vLLM Planner Matrix

Table 1 reports the four completed Path C cells. Values are shown in GiB to keep the table readable; exact byte values appear in the supplement and committed matrix artifact.

Interpretation. The observed planner artifacts reserve substantially more than the useful hybrid cache payload in every cell. The same ratio appears across sequence length and memory-utilization settings, which suggests a cache-shape accounting effect rather than a one-off scheduler anomaly.

4.2 Planner-Only RTX 3060 Comparison

The second track isolates cache-shape accounting in a deterministic synthetic harness. It compares a uniform padded planner model against a native KV/state planner model for three hybrid KV/state workloads. The target profile is an NVIDIA RTX 3060 with 12,884,901,888 target bytes.

Interpretation. Figure 2 makes the planner distinction visible. In the synthetic pack, the native KV/state model estimates close to useful bytes for all three workloads. The uniform padded model ranges from 2.868770x to 3.934376x. The large workloads still have virtual OOM under both planner models because the useful demand itself exceeds the target profile; this is a capacity result, not a failure of the native accounting model.

4.3 Evidence Boundary

The two tracks support the same narrow claim: hybrid cache over-reservation is visible in planner artifacts, and a native KV/state planner model can avoid the padded planner's accounting overhead in deterministic planner replay. They do not measure live serving memory, request admission, latency, throughput, quality, or accuracy. They also do not establish controlled substitution readiness, because the observed run does not resolve Mamba state-index and state tensor contracts.

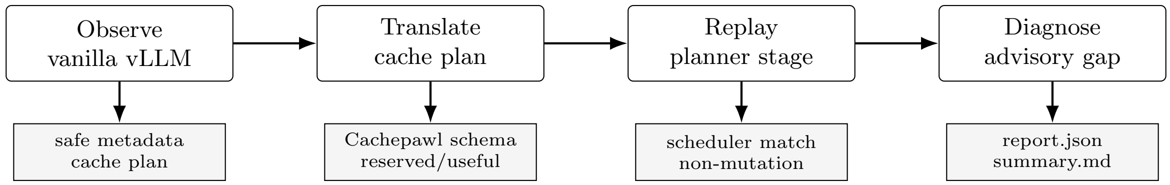

Figure 1: Read-only Path C workflow. The workflow converts captured vLLM planner artifacts into advisory metrics without returning a Cachepawl plan to the serving runtime.

Workload	Planner model	Useful GiB	Estimated GiB	Ratio	Waste	Virtual OOM
short-heavy	uniform padded	2.27	6.51	2.868770	0.651418	no
short-heavy	native KV/state	2.27	2.28	1.006567	0.006524	no
long-heavy	uniform padded	39.09	153.78	3.934376	0.745830	yes
long-heavy	native KV/state	39.09	39.10	1.000384	0.000384	yes
mixed	uniform padded	12.59	47.79	3.795917	0.736559	yes
mixed	native KV/state	12.59	12.60	1.001108	0.001107	yes

Table 2: Deterministic RTX 3060 planner-only comparison. Exact byte values are preserved in the supplement and artifact pack.

5 Limitations

The observed matrix covers one model, one vLLM version, two sequence lengths, two memory-utilization settings, and `max_num_seqs=1`. The RTX 3060 pack is synthetic and planner-only. These limits are appropriate for a diagnostic paper, but they bound the claims. A stronger systems paper would need multi-model evidence, controlled mutation hooks, Mamba state contracts, and live serving measurements.

6 Conclusion

Hybrid Attention/Mamba serving exposes an accounting problem before it exposes an allocator replacement problem. Cachepawl Path C shows that planner-level over-reservation can be diagnosed from committed vLLM planner artifacts without changing runtime state. The workshop contribution is a clean diagnostic workflow, a bounded vLLM planner matrix, and a deterministic planner-only comparison that separates supported advisory evidence from future runtime integration work.

References

[1] Paolo Glorioso, Quentin Anthony, Yury Tokpanov, Anna Golubeva, Vasudev Shyam, James Whittington, Jonathan Pilault, and Beren Millidge. The Zamba2 suite: Technical report. *arXiv preprint arXiv:2411.15242*, 2024.

[2] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP '23)*. ACM, 2023.

[3] Opher Lieber, Barak Lenz, Hofit Bata, Gal Cohen, Jhonathan Osin, Itay Dalmedigos, Erez Safahi, Shaked Meirom, Yonatan Belinkov, Shai Shalev-Shwartz, Omri Abend, Raz Alon, Tomer Asida, Amir Bergman, Roman Glozman, Michael Gokhman, Avshalom Manevich, Nir Ratner, Noam Rozen, Erez Shwartz, Mor Zusman, and Yoav Shoham. Jamba: A hybrid transformer-mamba language model. *arXiv preprint arXiv:2403.19887*, 2024.

[4] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

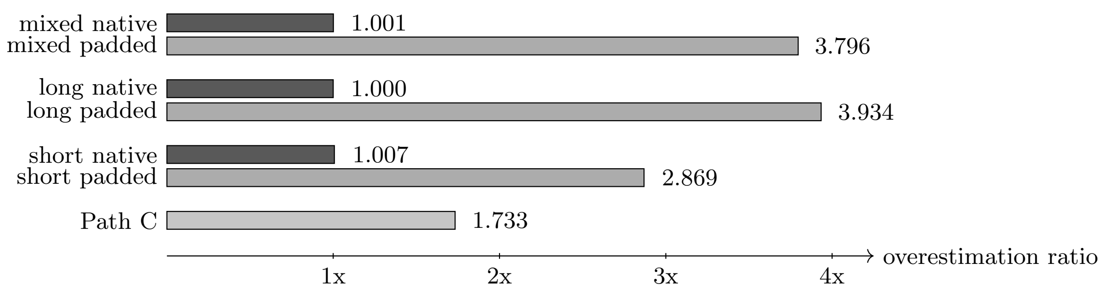

Figure 2: Overestimation ratios in the observed Path C matrix and the deterministic RTX 3060 planner-only comparison. The native KV/state planner model stays near useful bytes in the synthetic pack, while the padded model reserves substantially more.